

PROJECT WORK REPORT

Legal RAG System — AI-Powered Due Diligence Platform

Internship Project · Pair A · AI/ML Domain · 5-Day Sprint

Submitted by (Group Partners):

Ahmed Qureshi

Abdul Ghafoor

Reporting Period: One-week sprint

Total Team Effort: 28 person-hours (approx.)

Date: July 14, 2026

1. Executive Summary

This report documents the work completed by group partners Ahmed Qureshi and Abdul Ghafoor over a one-week sprint on the Legal RAG System project — an AI-powered due-diligence platform built for Pakistani law firms. The objective of the project is to compress a legal document review process that normally takes 5-7 days down to same-day turnaround, using a Retrieval-Augmented Generation (RAG) pipeline purpose-built for legal clause review.

Based on the scope, complexity, and volume of the deliverables produced this week — a working end-to-end backend pipeline, a 52-component frontend, LLM-driven memo generation, red-flag detection, and a full supporting business case (pricing, cost analysis, architecture documentation, pitch deck, and UI design) — the combined effort by both partners this week is estimated at approximately 28 person-hours, consistent with the team's own estimate of 25+ hours. Section 4 breaks this down task-by-task against the artifacts actually present in the submitted project files.

Note on hour tracking: the project folder does not contain a connected time-tracking log or Git commit history, so the figures below are a reconstructed estimate — built by matching each hour block to a specific, verifiable deliverable in the codebase and documentation, rather than an arbitrary total. Where a task's real effort could reasonably vary, a conservative mid-range estimate was used.

2. Project Overview

2.1 Problem Statement

Legal due-diligence review at Pakistani law firms (property, loan, and acquisition transactions) is currently a manual, 5-7 day process performed by senior associates, at a stated cost of roughly PKR 250,000 per week in senior associate time per firm.

2.2 Solution Built

A Retrieval-Augmented Generation system that ingests a bundle of legal PDFs (Sale Deed, Registry, NOC, Mutation, Tax Certificate, etc.), automatically extracts and OCRs the content (including mixed English/Urdu land records), indexes it, retrieves the most relevant clauses against a fixed due-diligence checklist, and uses an LLM to generate a structured, citation-backed due-diligence memo — flagging high-risk findings automatically.

2.3 Technology Stack

- Backend orchestration: FastAPI (Python 3.11+)
- RAG orchestration: LlamaIndex (SentenceSplitter chunking, VectorStoreIndex)
- Embeddings: HuggingFace sentence-transformers/all-MiniLM-L6-v2 (local, zero API cost)
- Vector store: ChromaDB, one isolated collection per client session
- OCR: pdf2image + pytesseract with English + Urdu language support
- LLM generation: Claude Sonnet 4.6 in production, Groq Llama 3.3 70B for development/testing
- Frontend: React + Vite, with a 52-component UI library (shadcn/ui-based)
- Output generation: python-docx and ReportLab for Word/PDF memo delivery

3. Scope of Work Completed

The following components exist as working, verifiable artifacts in the submitted project files:

3.1 Document Ingestion Pipeline (backend/services/document_processor, ocr)

- PDF validation layer — extension, file size, and corruption/password checks (validators.py) before any heavy processing runs.

- Native text extraction via pdfplumber, with automatic OCR fallback triggered per-page when extracted text falls below a minimum character threshold.
- OCR service using pdf2image + pytesseract configured for combined English + Urdu recognition, needed for bilingual Pakistani land records (Fard, Intiqal, Mutation).
- Text-cleaning utility that removes OCR artifacts and rejoins broken line-wraps, while deliberately preserving numbered clause/section boundaries so legal structure is not lost.

3.2 Chunking, Embedding, and Vector Storage

- Per-page chunking using LlamaIndex SentenceSplitter (chunk_size=512, chunk_overlap=50) so every chunk keeps accurate page-number metadata for later citation.
- Local embedding service using all-MiniLM-L6-v2 — no external API calls, satisfying client-confidentiality requirements for unreleased legal documents.
- Session-isolated ChromaDB collections (one per upload session, named by UUID) to prevent cross-contamination between concurrent client engagements.
- LlamaIndex VectorStoreIndex build/persist/reload logic so an index survives between requests without re-embedding.

3.3 Retrieval, Memo Generation, and Red-Flag Detection

- Retrieval-only query engine returning typed, scored chunks (filename, page number, similarity score, text) — kept as a separate module specifically so generation could be swapped in without touching ingestion.
- LLM integration service (Groq Llama 3.3 70B, OpenAI-compatible interface) producing structured JSON output per finding: finding, clause reference, page number, risk level, recommendation.
- Red-flag detector (259 lines) that scans generated findings and automatically surfaces any HIGH risk-level item.
- Checklist service driving the 15-question due-diligence checklist used to structure retrieval per transaction type (property / loan / acquisition).
- Memo generator and memo router assembling retrieved findings into a structured due-diligence memo.

3.4 API Layer

- FastAPI endpoint (app.py) exposing POST /ingest and POST /query/{session_id}, with CORS enabled for the React dev server.
- Ingestion and memo routers, Pydantic request/response schemas, and centralized logging across every pipeline stage.
- A CLI retrieval tool (query.py) for direct testing of the pipeline without the frontend.

3.5 Frontend Application

- React + Vite application with 52 component files, built on a shadcn/ui-based component library (dialogs, tables, forms, tabs, sidebar, command palette, and more).
- Multi-file drag-and-drop upload flow, transaction-type selection, and a live pipeline progress indicator.
- Download flow for the completed Word/PDF memo output.

3.6 Output Generation

- python-docx assembly of the Word memo (cover page, transaction summary, clause-by-clause findings table, red-flags section, missing-documents checklist, recommendations) with firm letterhead injection.
- ReportLab PDF generation as the second output format.

3.7 Business Case and Supporting Documentation

- Technology-stack justification document comparing LlamaIndex vs. LangChain, and Claude Sonnet 4.6 vs. Groq Llama 3.3 70B, with cost and performance reasoning for each choice.
- Nine-layer technical architecture write-up covering the full pipeline from UI to output generation.
- A three-tier pricing model (Starter / Professional / Enterprise, priced in PKR) with an ROI justification for the Professional tier.
- A cost-of-inaction analysis quantifying revenue loss, wasted senior-associate time, error-liability exposure, and client-attrition risk for firms that do not adopt the system.
- A cost-breakdown spreadsheet and a separate cost-analysis spreadsheet.
- A pitch deck (Law Firm Rag Model Pitch.pptx) for project presentation.
- A Figma UI design file and a Loom walkthrough video, referenced in the submitted links.

4. Detailed Hours Breakdown

The table below allocates the week's effort across every task area for which a working deliverable exists in the submitted project. This is a joint total for both group partners (Ahmed Qureshi and Abdul Ghafoor) working together across backend, frontend, and business-case workstreams; the project files do not separately log which partner performed which task, so hours are reported at the team level rather than split per individual.

Task Area	Work Completed (Evidence)	Hours
Requirements & architecture planning	9-layer technical architecture; stack selection & justification (LlamaIndex vs. LangChain, Claude vs. Groq)	2.5
Document ingestion pipeline	PDF validation, pdfplumber extraction, automatic OCR fallback (English + Urdu)	4.0
Chunking & embedding	Per-page LlamaIndex SentenceSplitter chunking; local all-MiniLM-L6-v2 embedding service	2.5
Vector store & indexing	Session-isolated ChromaDB collections; VectorStoreIndex persistence/reload	2.0
Retrieval / query engine	Typed, scored chunk retriever decoupled from generation	1.5
LLM memo generation & red-flag detection	Groq/Claude structured JSON generation; 259-line red-flag detector; 15-item checklist service	4.0
FastAPI backend & routers	app.py endpoint, ingestion & memo routers, Pydantic schemas, logging & validators	2.5
Frontend (React + Vite)	52-component UI, upload flow, transaction-type selection, progress tracking	4.5
Output generation	python-docx / ReportLab memo assembly with letterhead	1.5
Business case documentation	Pricing model, cost-of-inaction analysis, cost-breakdown spreadsheets	1.5
Pitch deck, UI design & demo	Pitch pptx, Figma design file, Loom walkthrough	1.5
Testing, debugging & integration	End-to-end pipeline testing against sample PDFs, log review, bug fixes	1.5
Total		28.0

Total: approximately 28 person-hours for the week, consistent with the team's reported 25+ hours.

5. Summary of Deliverables

- Working FastAPI backend implementing the full ingestion → OCR → chunk → embed → index → retrieve pipeline
- LLM-driven memo generation service with structured, cited findings and automatic red-flag detection
- React + Vite frontend (52 components) for upload, review, and memo download
- Word and PDF memo output generation with firm letterhead
- Technical architecture and technology-stack justification documents
- Three-tier pricing model and cost-of-inaction business case, with supporting spreadsheets
- Pitch deck, Figma design file, and Loom demo walkthrough
- Project GitHub repository containing all of the above

6. Conclusion

Over the course of this one-week sprint, group partners Ahmed Qureshi and Abdul Ghafoor jointly delivered a complete, working prototype of the Legal RAG System — spanning document ingestion, OCR, embedding, retrieval, LLM-based memo generation, a full frontend application, and the accompanying business case and presentation materials. The estimated 28 person-hours of combined effort documented above is directly traceable to the artifacts present in the submitted project files and is consistent with the team's reported 25+ hours worked this week.